❑ 개념 확인

(1) **[생성형 AI 활용]** 스레드 객체를 생성하는 방법에 대하여 예와 함께 설명하세요

생성형 AI	
질문	
답변	

(2) **[생성형 AI 활용]** 문자열 분리 방법에 대하여 예와 함께 설명하세요

생성형 AI	
질문	
답변	

(3) 정수 문자열을 입력 받은 후 소수의 합을 계산하여 출력 하세요. 문자열 분리는 split() 메소드를 사용합니다. 단, 구분자로 콤마를 사용하며 제시된 main()을 사용합니다.

```
정수와 정수 사이는 ,(콤마)로 구분하세요
>> 11,31,24,53,35,78,49,55,36,96,374,35
11
31
24
53
35
78
49
55
36
96
374
35
소수 :  11  31  53
sum = 95
```

```
public static void main(String[] args) {
    Scanner key = new Scanner(System.in);
    Calc cal=null;

    System.out.print("정수 문자열을 입력하세요. ₩n 정수와 정수 사이는 ,(콤마)로 구분하세요 ₩n>> ");
    String digit_str = key.nextLine();
    cal=new Calc(digit_str);
    cal.show();
    System.out.printf("sum = %d₩n", cal.sum());
}
```

[프로그램 소스]

```
import java.util.*;
class Calc{
    private String[] att;
    public Calc(String str) {
        att=str.split(",");
    }
    public void show() {
        for(String item : att) {
            System.out.println(item);
        }
```

```
        }

        public static boolean isPrime(int n) {
        if (n <= 1)
                return false; // 1 이하의 수는 소수가 아님
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0)
                return false; // 나누어 떨어지면 소수가 아님
        }
        return true; // 나누어 떨어지는 경우가 없으면 소수
    }

        public int sum() {
                int total=0;
                String message="소수 : ";
                for(String item : att) {
                        int value=Integer.parseInt(item);
                        if(isPrime(value)) {
                                message +="   "+value;
                                total += value;
                        }
                }
                System.out.println(message);
                return total;
        }
}
public class Test {
        public static void main(String[] args) {
                Scanner key = new Scanner(System.in);
                Calc cal=null;

                System.out.print("정수 문자열을 입력하세요. ₩n 정수와 정수 사이는 ,(콤마)로 구분하세요
₩n>> ");
                String digit_str = key.nextLine();
                cal=new Calc(digit_str);
                cal.show();
                System.out.printf("sum = %d₩n", cal.sum());
        }
}
```

[실행 결과]

(4) 다음과 같이 특수 문자와 영문자, 숫자를 출력하는 스레드를 작성하고 테스트 하는 프로그램을 작성하세요. 단, 출력시 0.5초의 간격이 있도록 합니다. 특수 문자와 영문자, 숫자 구간은 본인이 임의로 결정합니다. 스레드 객체는 람다식, 익명자식객체, 익명구현객체를 각각 사용합니다.

```
특수문자: ◆
숫자: 1
영문자: F
영문자: G
숫자: 2
특수문자: ▣
숫자: 3
영문자: H
특수문자: ※
숫자: 4
영문자: I
특수문자: ○
영문자: J
특수문자: ♠
숫자: 5
특수문자: ★
영문자: K
특수문자: ♣
```

```
public class TestThread {
        public static void main(String[] args) {


        }
}
```

[프로그램 소스]

```
public class TestThread {
    public static void main(String[] args) {
        // 1. 람다식 사용 (영문자 출력)
        Thread chThread= new Thread(() -> {
            for (char c = 'F'; c <= 'K'; c++) {
                System.out.println("영문자: " + c);
                try {
                    Thread.sleep(500); // 0.5 초 대기
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });

        // 2. 익명 자식 객체 사용 (숫자 출력)
        Thread numThread = new Thread() {
            @Override
            public void run() {
                for (int i = 1; i <= 5; i++) {
                    System.out.println("숫자: " + i);
                    try {
                        Thread.sleep(500); // 0.5 초 대기
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
            }
```

```
        };

        // 3. 익명 구현 객체 사용 (특수문자 출력)
        Thread speThread = new Thread(new Runnable() {
            @Override
            public void run() {
                char[] specials = {'◆', '▣', '※', '○', '♠', '★', '♣'};
                for (char c : specials) {
                    System.out.println("특수문자: " + c);
                    try {
                        Thread.sleep(500); // 0.5 초 대기
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
            }
        });

        // 스레드 시작
        chThread.start();
        numThread.start();
        speThread.start();
    }
}
```

[실행 결과]

(5) 다음과 같은 배열 원소를 출력하는 각각의 스레드 객체를 생성하고 실행하는 프로그램을 작성하세요. 단, 상속과 인터페이스 구현 두 가지 방법으로 제시하세요

```
int-thread : 3
string-thread : hallym
string-thread : software
int-thread : 14
string-thread : big
int-thread : 52
string-thread : content
int-thread : 7
string-thread : IoT
int-thread : 11
string-thread : data
int-thread : 8
int-thread : 90
int-thread : 74
```

```
int-thread : 3
string-thread : hallym
int-thread : 14
string-thread : software
int-thread : 52
string-thread : big
int-thread : 7
string-thread : content
int-thread : 11
string-thread : IoT
int-thread : 8
string-thread : data
int-thread : 90
int-thread : 74
```

```
int[] intArray = {3,14,52,7,11,8, 90,74};
String[] strArray = {"hallym", "software", "big", "content", "IoT", "data"};
```

[프로그램 소스]

>> 인터페이스 구현

```
class IntArrayPrinter implements Runnable {
    private int[] array;
    private String name;

    public IntArrayPrinter(int[] array, String name) {
```

```
        this.array = array;
        this.name=name;
    }

    @Override
    public void run() {
        for (int num : array) {
            System.out.println(name+" : " + num);
            try {
                Thread.sleep(500); // 0.5 초 대기
            } catch (InterruptedException e) {
                System.out.println(name +", Thread interrupted");
            }
        }
    }
}

class StrArrayPrinter implements Runnable {
    private String[] array;
    private String name;

    public StrArrayPrinter(String[] array, String name) {
        this.array = array;
        this.name=name;
    }

    @Override
    public void run() {
        for (String str : array) {
            System.out.println(name+" : " + str);
            try {
                Thread.sleep(500); // 0.5 초 대기
            } catch (InterruptedException e) {
                System.out.println(name +",     Thread interrupted");
            }
        }
    }
}

public class Test {
        public static void main(String[] args) {
                int[] intArray = {3,14,52,7,11,8, 90,74};
                String[] strArray = {"hallym", "software", "big", "content", "IoT", "data"};
```

```
            Thread intArrayThread = new Thread(new IntArrayPrinter(intArray, "int-thread"));
            Thread strArrayThread = new Thread(new StrArrayPrinter(strArray, "string-thread"));

            intArrayThread.start();
            strArrayThread.start();
        }
}
```

>> 상속 구현

```
class IntArrayPrinter extends Thread {
    private int[] array;

    public IntArrayPrinter(int[] array, String name) {
        this.array = array;
        setName(name);
    }

    @Override
    public void run() {
        for (int num : array) {
            System.out.println(getName()+" : " + num);
            try {
                Thread.sleep(500); // 0.5 초 대기
            } catch (InterruptedException e) {
                System.out.println(getName() +", Thread interrupted");
            }
        }
    }
}

class StrArrayPrinter extends Thread {
    private String[] array;

    public StrArrayPrinter(String[] array, String name) {
        this.array = array;
        setName(name);
    }

    @Override
    public void run() {
        for (String str : array) {
            System.out.println(getName()+" : " + str);
            try {
                Thread.sleep(500); // 0.5 초 대기
```

```
                } catch (InterruptedException e) {
                        System.out.println(getName() +",     Thread interrupted");
                }
        }
    }
}

public class Test {
        public static void main(String[] args) {
                int[] intArray = {3,14,52,7,11,8, 90,74};
                String[] strArray = {"hallym", "software", "big", "content", "IoT", "data"};

                Thread intArrayThread = new IntArrayPrinter(intArray, "int-thread");
                Thread strArrayThread = new StrArrayPrinter(strArray, "string-thread");

                intArrayThread.start();
                strArrayThread.start();
        }
}
```

[실행 결과]

❑ 응용 프로그래밍

(6) 현재 날짜와 시간을 출력한 후, 현재를 기준으로 5일후의 날짜도 함께 출력하세요. 단, time 패키지 사용

[프로그램 소스]

```
 public static void main(String[] args) {
                LocalDateTime now = LocalDateTime.now();
                String strDateTime = now.getYear() + "년 ";
                strDateTime += now.getMonthValue() + "월 ";
                strDateTime += now.getDayOfMonth() + "일 ";
                strDateTime += now.getDayOfWeek() + " ";
                strDateTime += now.getHour() + "시 ";
                strDateTime += now.getMinute() + "분 ";
                strDateTime += now.getSecond() + "초 ";

                System.out.println("=======현재시 ======= ");
                System.out.println(strDateTime);
                LocalDateTime   targetDateTime = now.plusDays(50);
                String str = targetDateTime.getYear() + "년 ";
                str += targetDateTime.getMonthValue() + "월 ";
                str += targetDateTime.getDayOfMonth() + "일 ";
                System.out.println("₩n======= 5일 후 ===== ");
```

```
                System.out.println(str);
}
```

[실행 결과]

(7) 제시된 main() 메소드와 결과를 참고하여 다음 설명에 해당하는 프로그램을 작성하세요.

1. Thread를 상속받은 Job 클래스를 작성하세요.

>> 필드(private String name, int num)

>> 생성자(이름을 전달하여 저장)

>> run() 메소드 : 0.5초간 sleep() 후 getNum()메소드 호출, getNum()에서 리턴 받은 값이 10이면 반복 탈출

>> getNum() : 한 순간에 하나의 스레드만 접근할 수 있는 메소드, num값을 1증가 시킨 후 증가된 num값을 리턴

```
public class Test{
    public static void main(String[] args) {
        Thread t1 = new Job("ThreadA");
        Thread t2 = new Job("ThreadB");
        t1.start();
        t2.start();
    }
}
```

```
ThreadB: 1
ThreadA: 1
ThreadB: 2
ThreadA: 2
ThreadB: 3
ThreadA: 3
ThreadB: 4
ThreadA: 4
ThreadB: 5
ThreadA: 5
ThreadB: 6
ThreadA: 6
ThreadB: 7
ThreadA: 7
ThreadB: 8
ThreadA: 8
ThreadB: 9
ThreadA: 9
```

[프로그램 소스]

```
class Job extends Thread {
        private String name;
        int num = 0;

        public Job(String name) {
                this.name = name;
        }

        public void run() {
                while (true) {
                        try {
                                Thread.sleep(500);
                                int n = getNum();
                                if (n == 10)
                                        break;
                                System.out.println(name + ": " + n);
                        } catch (InterruptedException e) {
                                System.out.println(e.toString());
                        }
                }
        }
        public synchronized int getNum() {
```

```
                return ++num;
        }
}

public class Test {
        public static void main(String[] args) {
                Thread t1 = new Job("ThreadA");
                Thread t2 = new Job("ThreadB");
                t1.start();
                t2.start();
        }
}
```

[실행 결과]

(8) 11장 수업자료 31번 슬라이드에 제시된 예제 프로그램을 다음과 같이 동작될 수 있도록 수정하세요

```
public class WaitNofifyExam {
    public static void main(String[] args) {
            MessageBox dataBox = new MessageBox();
            String[] message = new String[] { "hallym", "big data", "software", "AI" };

            SendThread sendThread = new SendThread(dataBox, message);
            ReceiveThread receiveThread = new ReceiveThread(dataBox, message.length);

            sendThread.start();
            receiveThread.start();
    }
}
```

```
SendThread 스레드가 전송한 데이터: hallym
ReceiveThread 스레드가 수신한 데이터 : hallym
SendThread 스레드가 전송한 데이터: big data
ReceiveThread 스레드가 수신한 데이터 : big data
SendThread 스레드가 전송한 데이터: software
ReceiveThread 스레드가 수신한 데이터 : software
SendThread 스레드가 전송한 데이터: AI
ReceiveThread 스레드가 수신한 데이터 : AI
```

[프로그램 소스]

```
class MessageBox {
        private String data;

        public synchronized String receiveData() {
                if (this.data == null) {
                        try {
                                wait();
                        } catch (InterruptedException e) {
                        }
                }
                String returnValue = data;
```

```
                System.out.println(Thread.currentThread().getName() +" 스레드가 수신한 데이터 : " +
returnValue);
                data = null;
                notify();
                return returnValue;
        }

        public synchronized void sendData(String data) {
                if (this.data != null) {
                        try {
                                wait();
                        } catch (InterruptedException e) {
                        }
                }
                this.data = data;
                System.out.println(Thread.currentThread().getName()+ " 스레드가 전송한 데이터: " + data);
                notify();
        }
}

class ReceiveThread extends Thread {
        private MessageBox message;
        private int cnt;

        public ReceiveThread(MessageBox message, int cnt) {
                this.message = message;
                this.cnt = cnt;
                setName("ReceiveThread");
        }

        @Override
        public void run() {
                for (int i = 1; i <= cnt; i++) {
                        String data = message.receiveData();
                }
        }
}

class SendThread extends Thread {
        private MessageBox message;
        private String[] content;

        public SendThread(MessageBox message, String[] content) {
                this.message = message;
```

```
                this.content = content;
                setName("SendThread");
        }

        @Override
        public void run() {
                for (int i = 0; i < content.length; i++) {
                        message.sendData(content[i]);
                }
        }
}

public class WaitExam {
        public static void main(String[] args) {
                MessageBox dataBox = new MessageBox();
                String[] message = new String[] { "hallym", "big data", "software", "AI" };

                SendThread sendThread = new SendThread(dataBox, message);
                ReceiveThread receiveThread = new ReceiveThread(dataBox, message.length);

                sendThread.start();
                receiveThread.start();
        }
}
```

[실행 결과]

(9) 다음과 같이 문자열을 처리하는 프로그램을 작성하세요.

① StringTokenizer 클래스를 이용하여 사용자로부터 받은 문자열을 단어로 분리합니다. **단, 공백이 포함된 문자열 입력은 nextLine() 사용**

② 단어들을 문자열 배열에 넣고 이 배열을 Arrays의 sort() 메소드를 이용하여 정렬합니다

```
문자열을 입력하세요 :  Did you sleep well last night?
정렬 전 결과 :
Did     you     sleep     well     last     night?
정렬 후 결과 :
Did     last     night?     sleep     well     you
```

[프로그램 소스]

```
import java.util.*;
public class Test {
        public static void main(String[] args) {
                Scanner key=new Scanner(System.in);
                String[] str;
```

```
        int i=0;
        StringTokenizer st=null;

        System.out.print("문자열을 입력하세요 :    ");
        st = new StringTokenizer(key.nextLine());
        str = new String[st.countTokens()];

        while (st.hasMoreTokens()) {    //토큰이 있는 동안만
            str[i++] = st.nextToken();  //토큰을 추출하여
        }

        System.out.println("정렬 전 결과 :");
        for(String temp : str)
            System.out.print(temp+"        ");

        System.out.println("₩n정렬 후 결과 :");
        Arrays.sort(str);
        for(String temp : str)
            System.out.print(temp + "        ");
        System.out.println();
    }
}
```

[실행결과]